DBMS – ADVANCED SQL QUERY IMPLEMENTATION

ShopSphere – Online E-Commerce Database Management System

OBJECTIVES

1. To implement subqueries and nested queries.
2. To find products above the average price.
3. To identify customers with maximum purchases.
4. To generate complex business queries.
5. To prepare an advanced SQL report.

INTRODUCTION

ShopSphere is an Online E-Commerce Database Management System developed to manage customer details, product information, orders, payments and sales transactions. Advanced SQL queries are used to retrieve meaningful information from the database. Subqueries, nested queries, aggregate functions and joins help analyse product prices, customer purchases and business performance.

DATABASE TABLES USED

The following tables are used in the ShopSphere database:

1. Customer
2. Product
3. Orders
4. Order_Details
5. Category
6. Payment

These tables are connected through primary key and foreign key relationships to maintain the relationship between customers, products and orders.

1. IMPLEMENT SUBQUERIES AND NESTED QUERIES

A subquery is a query written inside another SQL query. The inner query executes first and provides the result to the outer query.

Subqueries are useful for retrieving data based on conditions obtained from another query.

Types of Subqueries:

- Single-row subquery – Returns a single value.
- Multiple-row subquery – Returns multiple values.
- Nested subquery – A query written inside another subquery.
- Correlated subquery – The inner query depends on the outer query.

Implementation:

In ShopSphere, subqueries are used to find orders above average order amount, customers who have placed orders and products based on specific price conditions.

SQL Query 1: Orders Above Average Order Amount

```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
  2       FROM Orders
  3       WHERE Total_Amount > (SELECT AVG(Total_Amount)
  4                             FROM Orders);

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
      1002         102         2499
      1004         104         3598
```

SQL Query 2: Customers Who Have Placed Orders

```
SQL> SELECT Customer_ID, Customer_Name
  2       FROM Customer
  3       WHERE Customer_ID IN (SELECT Customer_ID
  4                             FROM Orders);

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
        101 Arun Kumar
        102 Priya Sharma
        103 Rahul Raj
        104 Divya Sri
```

SQL Query 3: Products Above Average Price

```
SQL> SELECT Product_ID, Product_Name, Price
  2       FROM Product
  3       WHERE Price > (SELECT AVG(Price)
  4                      FROM Product);

PRODUCT_ID PRODUCT_NAME                                       PRICE
---------- -------------------------------------------------- ----------
       101 Laptop                                             45000
       104 Mobile Phone                                       18000
```

SQL Query 4: Customers Who Have Placed at Least One Order

```
SQL> SELECT C.Customer_ID, C.Customer_Name
  2       FROM Customer C
  3       WHERE EXISTS (SELECT 1
  4                     FROM Orders O
  5                     WHERE O.Customer_ID = C.Customer_ID);

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
        101 Arun Kumar
        102 Priya Sharma
        103 Rahul Raj
        104 Divya Sri
```

2. FIND PRODUCTS ABOVE AVERAGE PRICE

This query identifies products whose prices are greater than the average price of all products available in the ShopSphere database.

The average product price is calculated using the AVG() aggregate function. The result is compared with individual product prices to identify products above the average.

Purpose:

- To identify high-priced products.
- To compare product prices with the average price.
- To support product pricing analysis.
- To understand product price distribution.

```
SQL> SELECT Product_ID,
  2             Product_Name,
  3             Price
  4       FROM Product
  5       WHERE Price > (SELECT AVG(Price)
  6                      FROM Product)
  7       ORDER BY Price DESC;

PRODUCT_ID PRODUCT_NAME                                       PRICE
---------- -------------------------------------------------- ----------
       101 Laptop                                             45000
       104 Mobile Phone                                       18000
```

3. IDENTIFY CUSTOMERS WITH MAXIMUM PURCHASES

This query identifies the customer who has made the highest total purchase in the ShopSphere system.

The total purchase amount of each customer is calculated using the SUM() aggregate function. The maximum purchase amount is identified using the MAX() function.

Purpose:

- To identify the highest purchasing customer.
- To analyse customer purchasing behaviour.
- To identify valuable customers.
- To support customer relationship management.

```
SQL> SELECT C.Customer_ID,
  2             C.Customer_Name,
  3             SUM(O.Total_Amount) AS Total_Purchase
  4       FROM Customer C
  5       JOIN Orders O
  6       ON C.Customer_ID = O.Customer_ID
  7       GROUP BY C.Customer_ID, C.Customer_Name
  8       HAVING SUM(O.Total_Amount) =
  9       (SELECT MAX(Customer_Total)
 10        FROM (SELECT SUM(Total_Amount) AS Customer_Total
 11              FROM Orders
 12              GROUP BY Customer_ID));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_PURCHASE
----------- -------------------------------------------------- --------------
        104 Divya Sri                                                    3598
```

4. GENERATE COMPLEX BUSINESS QUERIES

Complex business queries combine multiple tables, joins, subqueries and aggregate functions to retrieve useful business information.

4.1 Customers Who Placed Orders Above Average Order Value

This query identifies customers who have placed orders with an amount greater than the average order value.

Purpose:

- To identify customers making high-value orders.
- To analyse customer spending behaviour.
- To understand purchasing patterns.

```
SQL> SELECT DISTINCT C.Customer_ID,
  2                      C.Customer_Name
  3       FROM Customer C
  4       JOIN Orders O
  5       ON C.Customer_ID = O.Customer_ID
  6       WHERE O.Total_Amount > (SELECT AVG(Total_Amount)
  7                               FROM Orders)
  8       ORDER BY C.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME
----------- --------------------------------------------------
        102 Priya Sharma
        104 Divya Sri
```

4.2 Products Whose Total Sales Are Above Average

This query identifies products whose total sales are greater than the average sales generated by all products.

Purpose:

- To identify products generating higher sales.
- To analyse product performance.
- To support business sales decisions.

```
SQL> SELECT P.Product_ID,
  2             P.Product_Name,
  3             SUM(OD.Quantity * OD.Unit_Price) AS Total_Sales
  4       FROM Product P
  5       JOIN Order_Details OD
  6       ON P.Product_ID = OD.Product_ID
  7       GROUP BY P.Product_ID, P.Product_Name
  8       HAVING SUM(OD.Quantity * OD.Unit_Price) >
  9       (SELECT AVG(Product_Sales)
 10        FROM (SELECT SUM(Quantity * Unit_Price) AS Product_Sales
 11              FROM Order_Details
 12              GROUP BY Product_ID))
 13       ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_SALES
---------- -------------------------------------------------- -----------
       104 Mobile Phone                                              4947
       102 Mouse                                                     2499
```

5. PREPARE AN ADVANCED SQL REPORT

Advanced SQL reports combine information from multiple database tables to generate detailed business reports.

5.1 Customer Purchase Report

The customer purchase report displays customer information along with their purchase details.

Report Contains:

- Customer ID
- Customer Name
- Number of Orders
- Total Purchase Amount
- Average Order Value

Purpose:

This report helps the management understand customer purchasing behaviour and identify customers who contribute more to the business.

```
SQL> SELECT C.Customer_ID,
  2              C.Customer_Name,
  3              COUNT(O.Order_ID) AS Number_Of_Orders,
  4              NVL(SUM(O.Total_Amount), 0) AS Total_Purchase,
  5              NVL(ROUND(AVG(O.Total_Amount), 2), 0) AS Average_Order_Value
  6       FROM Customer C
  7       LEFT JOIN Orders O
  8       ON C.Customer_ID = O.Customer_ID
  9       GROUP BY C.Customer_ID, C.Customer_Name
 10       ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME                                      NUMBER_OF_ORDERS
----------- -------------------------------------------------- ----------------
TOTAL_PURCHASE AVERAGE_ORDER_VALUE
-------------- -------------------
        104 Divya Sri                                                         1
          3598               3598

        102 Priya Sharma                                                      1
          2499               2499

        101 Arun Kumar                                                        1
          1499               1499


CUSTOMER_ID CUSTOMER_NAME                                      NUMBER_OF_ORDERS
----------- -------------------------------------------------- ----------------
TOTAL_PURCHASE AVERAGE_ORDER_VALUE
-------------- -------------------
        103 Rahul Raj                                                         1
           899                899

        105 Karthik S                                                         0
             0                  0
```

5.2 Advanced Product Sales Report

The advanced product sales report provides detailed information about product sales and order transactions.

Report Contains:

- Product ID
- Product Name
- Number of Orders
- Total Quantity Sold
- Total Sales
- Average Selling Price

Purpose:

This report helps identify popular products, analyse sales performance and understand the revenue generated by individual products.

```
SQL> SELECT P.Product_ID,
  2              P.Product_Name,
  3              COUNT(DISTINCT OD.Order_ID) AS Number_Of_Orders,
  4              NVL(SUM(OD.Quantity), 0) AS Total_Quantity_Sold,
  5              NVL(SUM(OD.Quantity * OD.Unit_Price), 0) AS Total_Sales,
  6              NVL(ROUND(AVG(OD.Unit_Price), 2), 0) AS Average_Selling_Price
  7       FROM Product P
  8       LEFT JOIN Order_Details OD
  9       ON P.Product_ID = OD.Product_ID
 10       GROUP BY P.Product_ID, P.Product_Name
 11       ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME                                       NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------------- ----------- --------------------
       104 Mobile Phone                                                     1
                 3        4947                 1649

       102 Mouse                                                            1
                 1        2499                 2499

       101 Laptop                                                           1
                 1        1499                 1499


PRODUCT_ID PRODUCT_NAME                                       NUMBER_OF_ORDERS
---------- -------------------------------------------------- ----------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SELLING_PRICE
------------------- ----------- --------------------
       103 Keyboard                                                         1
                 1         899                  899

       105 Headphones                                                       0
                 0           0                    0
```

5.3 Category-Wise Advanced Sales Report

This report displays category-wise sales information by combining Category, Product and Order_Details tables.

Report Contains:

- Category ID
- Category Name
- Total Products
- Total Quantity Sold
- Total Sales

Purpose:

This report helps management compare the sales performance of different product categories and identify categories generating higher revenue.

```
SQL> SELECT C.Category_ID,
  2              C.Category_Name,
  3              COUNT(DISTINCT P.Product_ID) AS Total_Products,
  4              NVL(SUM(OD.Quantity), 0) AS Total_Quantity_Sold,
  5              NVL(SUM(OD.Quantity * OD.Unit_Price), 0) AS Total_Sales
  6       FROM Category C
  7       JOIN Product P
  8       ON C.Category_ID = P.Category_ID
  9       LEFT JOIN Order_Details OD
 10       ON P.Product_ID = OD.Product_ID
 11       GROUP BY C.Category_ID, C.Category_Name
 12       ORDER BY Total_Sales DESC;

CATEGORY_ID
-----------
CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_PRODUCTS TOTAL_QUANTITY_SOLD TOTAL_SALES
-------------- ------------------- -----------
         1
Electronics
             5                   6        9844
```

CONCLUSION

The Advanced SQL Query Implementation task was successfully completed using the ShopSphere Online E-Commerce Database Management System. Subqueries, nested queries, aggregate functions and joins were implemented to retrieve meaningful information from multiple database tables. The queries helped identify products above average price, customers with maximum purchases, products generating higher sales and detailed customer and product reports.